Indian Institute of Technology Palakkad

DS2020 Introduction To AI

Lab - 4

Submitted by:

Aarnava Rajan (142401003)

INDIAN INSTITUTE
OF TECHNOLOGY
PALAKKAD

1 Variable Encoding

To represent the Sudoku grid we define a set of boolean variables $x_{r,c,v}$ where r is the row index (0 -8) , c is the column index (0-8) and v is the value of the particular cell (1-9). Since Pycosat requires unique integer identifiers (excluding 0) we map these ($9 \times 9 \times 9 =$

729) possibilities using the following formula $ID = (r \times 9 + c) \times 9 + v$, which ensures each cell maps to a unique integer.

```
    def get_id(row,col,value):
        return (row * 9 + col) * 9 + value
```

2 CNF Encoding

2.1 Each cell contains atleast one digit

Each cell (r, c) must contain at least one digit from 1 - 9 .
$CNF : x_{r,c,1} \vee x_{r,c,2} .... \vee x_{r,c,9}$

```
for row in range(9):
            for col in range(9):
                clauses.append([get_id(row, col, v) for v in
                   range(1, 10)])
```

2.2 Each cell contains atmost one digit

A cell (r, c) cannot contain more than one digit. We use pairwise negation to enforce this.
$CNF : (\neg x_{r,c,v1} \vee \neg x_{r,c,v2})$ for all $v_1 < v_2$

```
 for row in range(9):
            for col in range(9):
                for v1 in range(1, 10):
                    for v2 in range(v1+1, 10):
                        clauses.append([-get_id(row, col, v1), -
                           get_id(row, col, v2)])
```

2.3 Each row contains every digit exactly once

Each row must contain every digit 1–9 exactly once. This can be achieved by combining at least once and at most once conditions.
CNF:
At most once : No two cells in the same row can contain the same digit

$$\neg x_{r,c_1,v} \vee \neg x_{r,c_2,v} \quad \forall\ c_1 < c_2$$

.
At least once: For every row r and digit v , at least one column must contain v

$$\bigvee_{c=0}^{8} x_{r,c,v}$$

```
for row in range(9):
            for v in range(1, 10):
                clauses.append([get_id(row, col, v) for col in range(9)])
                for col1 in range(9):
                    for col2 in range(col1+1, 9):
                        clauses.append([-get_id(row, col1, v), -get_id(row, col2, v)])
```

2.4 Each column contains every digit exactly once

Each column must contain every digit 1–9 exactly once. This can be achieved by combining at least once and at most once conditions.
CNF:
At most once : No two cells in the same row can contain the same digit

$$\neg x_{r_1,c,v} \vee \neg x_{r_2,c,v} \quad \forall\ r_1 < r_2$$

.
At least once: For every row c and digit v , at least one column must contain v

$$\bigvee_{r=0}^{8} x_{r,c,v}$$

```
for col in range(9):
            for v in range(1, 10):
                clauses.append([get_id(row, col, v) for row in range(9)])
                for row1 in range(9):
                    for row2 in range(row1+1, 9):
                        clauses.append([-get_id(row1, col, v), -get_id(row2, col, v)])
```

2.5 Each 3×3 box contains every digit exactly once

The grid is divided into nine 3×3 boxes. For box at block position (b_r, b_c), the cells are $(b_r \times 3 + r,\ b_c \times 3 + c)$ for $r, c \in \{0, 1, 2\}$.
CNF: *At least once:* At least one cell in the box must contain digit v:

$$\bigvee_{r=0}^{2} \bigvee_{c=0}^{2} x_{b_r \cdot 3+r,\ b_c \cdot 3+c,\ v}$$

At most once: No two cells in the same box can contain the same digit:

$$\neg x_{r_1,c_1,v} \vee \neg x_{r_2,c_2,v} \quad \forall\ (r_1, c_1) \neq (r_2, c_2)$$

```
for v in range(1, 10):
            for br in range(3):
                for bc in range(3):
                    box = [
                        get_id(br*3 + r, bc*3 + c, v)
                        for r in range(3) for c in range(3)
                    ]
                    clauses.append(box)
                    for i in range(len(box)):
                        for j in range(i+1, len(box)):
                            clauses.append([-box[i], -box[j]])
```

2.6 Unit clauses fixing the pre-filled given cells

The pre-filled clues provided in the input string are represented as unit clauses. If cell (r, c) is pre-filled with value n, we add the clause $(x_{r,c,n})$ to the solver.

```
for i, char in enumerate(line):
            if char.isdigit() and char != '0':
                r, c = i // 9, i % 9
                clauses.append([get_id(r, c, int(char))])
```

3 Implementation Details

For each puzzle, a fresh list of clauses is constructed encoding all six constraint groups as described in the Constraints section. The clauses are built and appended to a single clauses list that is passed directly to pycosat.solve().
The CNF formula is passed to pycosat.solve. The return value is checked using isinstance(result, list) — if it is a list, the puzzle is satisfiable and the solution is decoded; if it returns the string "UNSAT", the output for that puzzle is "No solution found".

The satisfying assignment returned by pycosat is a list of integers where positive literals represent true variables. For each positive literal lit, the digit and cell index are decoded as:

$$v = ((lit - 1) \bmod 9) + 1$$
$$\text{cell index} = \left\lfloor \frac{lit - 1}{9} \right\rfloor$$

The cell index directly maps to a position in the 81-character output string
Since the clauses list is re-initialized for every puzzle inside the loop, the solver handles multiple puzzles independently without any state leaking between puzzles. This makes the program correct for batch inputs of any size.